



Tesina 3

Analisi di Reti Neurali sul Dataset Wisconsin Breast Cancer

Gruppo A: *Gianmarco Luongo, Giada Pietrocola, Aurora Pisa, Giovanni Sivo*

Corso di Intelligenza Artificiale A.A. 2024/2025
Università degli Studi di Cassino e del Lazio Meridionale, Italia

Introduzione

L'identificazione precoce del cancro al seno è fondamentale per migliorare le prospettive di trattamento e sopravvivenza. Le tecniche di apprendimento automatico, in particolare le **reti neurali artificiali** (ANN), si sono dimostrate strumenti efficaci per la classificazione automatica delle masse tumorali. Tuttavia, le prestazioni di una rete neurale dipendono fortemente dalla scelta dei suoi parametri, come la configurazione architetturale, la funzione di attivazione, la strategia di ottimizzazione e i meccanismi di regolarizzazione.

In questo studio, viene analizzato l'impatto di tali parametri sulla capacità di classificazione di una rete neurale addestrata sul Wisconsin Breast Cancer Dataset. L'obiettivo è identificare le configurazioni ottimali della rete per migliorare la convergenza, la stabilità delle predizioni e la precisione diagnostica, bilanciando al contempo i vincoli computazionali.

Metodologie

Dataset Per l'analisi dei parametri delle reti neurali, è stato utilizzato il **Wisconsin Breast Cancer Dataset**, contenente 569 campioni di tumori al seno, ciascuno con 30 caratteristiche numeriche estratte da immagini di cellule tumorali ottenute tramite agoaspirato (FNA). La variabi-

le target è la classificazione binaria del tumore come maligno (M) o benigno (B).

Prima di procedere con l'addestramento del modello, è stata effettuata un'analisi preliminare del dataset per identificare eventuali problematiche. La distribuzione delle classi ha rivelato un dataset abbastanza bilanciato, con 357 campioni benigni e 212 maligni. Inoltre, non sono stati rilevati valori mancanti in nessuna delle colonne. Un altro aspetto importante analizzato sono stati gli outlier, identificati tramite il calcolo del *Z-score*. Complessivamente, il numero di outlier è relativamente moderato, suggerendo che, sebbene ci siano alcuni valori estremi, non sembrano essere prevalenti in modo significativo in tutto il dataset.

Una volta completata questa analisi, le caratteristiche numeriche sono state standardizzate utilizzando lo *StandardScaler* di scikit-learn per uniformare le scale delle variabili. Successivamente, il dataset è stato suddiviso in un set di addestramento e uno di test, con una divisione dell'80% per l'addestramento e il 20% per il test, per una valutazione adeguata delle performance del modello.

MLP Classifier Per l'implementazione della rete neurale, è stato scelto l'*MLPClassifier* di scikit-learn, un classificatore basato su una rete neurale multi-layer perceptron (MLP).

Nel corso dell'analisi, è stato inizialmente eseguito un esperimento utilizzando cross-

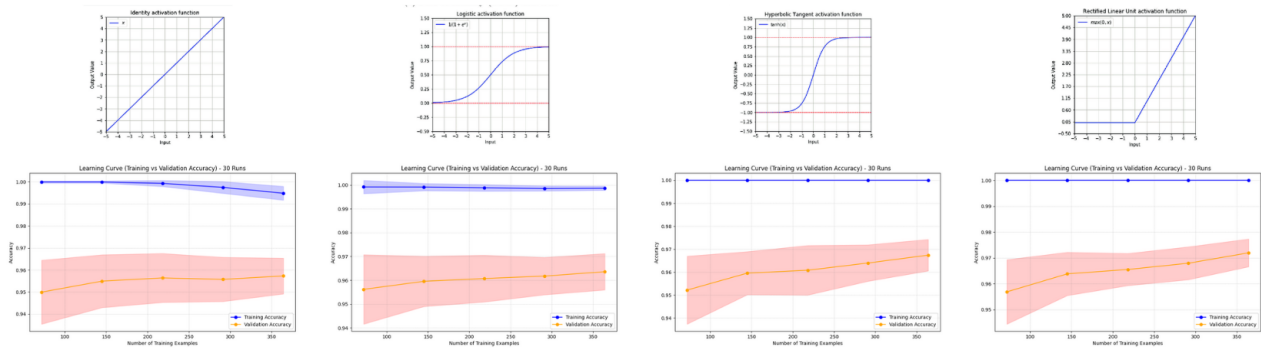


Figura 1: Curve di apprendimento per diverse funzioni di attivazione

validation con 5 fold per valutare la generalizzazione del modello, ottenendo una comprensione preliminare delle performance e della distribuzione delle metriche come l'accuratezza attraverso i diversi fold. Successivamente, per esplorare meglio la stabilità delle performance del modello e l'effetto della variabilità nei parametri di addestramento, sono state eseguite 30 run separate. In ciascuna run, il seed del generatore di numeri casuali è stato incrementato di uno, garantendo una riproducibilità dei risultati e la possibilità di analizzare la variabilità tra le esecuzioni.

Strategia di Validazione Per ogni run, sono stati monitorati e valutati diversi aspetti, tra cui l'accuratezza del modello, i tempi di training, le curve di apprendimento e la matrici di confusione mediate sulle 30 run, per analizzare gli errori di classificazione. Inoltre, sono state calcolate le curve ROC per ogni run e poi mediate, consentendo una valutazione delle prestazioni del modello in termini di sensibilità e specificità. È stata inoltre effettuata un'analisi della distribuzione delle accuratèzze per ogni run, rappresentando graficamente le performance individuali di ciascun esperimento. Infine, per esaminare la stabilità delle previsioni, è stato salvato un file CSV contenente le previsioni del modello, le classi vere e le probabilità di appartenenza a ciascuna classe per ogni campione nel set di validazione, permettendo di analizzare eventuali variazioni nelle previsioni e nella fiducia del modello.

Risultati

Funzioni di attivazione L'analisi delle funzioni di attivazione ha evidenziato differenze significative nelle prestazioni della rete neurale.

L'analisi ha mostrato che ReLU e tanh sono le funzioni di attivazione più efficaci per questo problema. **ReLU** ha ottenuto la migliore accuratezza di validazione, superando 0.97, con una training accuracy costantemente pari a 1. **tanh** ha raggiunto risultati simili ma con una maggiore variabilità tra le run. In termini di efficienza, ReLU ha richiesto meno tempo di addestramento (0.3s contro 0.5s di tanh), rendendola la scelta più vantaggiosa. Queste prestazioni superiori sono dovute al fatto che entrambe le funzioni introducono non linearità, permettendo alla rete di catturare meglio le relazioni complesse tra le feature.

Al contrario, **logistic** e **identity** hanno mostrato prestazioni inferiori. Tra le due logistic ha ottenuto una validation accuracy leggermente superiore, ma i suoi tempi di addestramento sono stati i più elevati (1.50s), a causa della saturazione dei gradienti che rallenta l'aggiornamento dei pesi. **identity**, essendo una funzione lineare, non introduce non linearità nel modello, limitando la capacità della rete di apprendere pattern complessi. Questo spiega la sua scarsa generalizzazione e il peggioramento della training accuracy con l'aumento dei dati, nonché il tempo di addestramento ridotto (0.2s), dato che il calcolo del gradiente è più semplice.

Configurazioni layer L'analisi delle diverse configurazioni di layer ha mostrato come la profondità della rete e la funzione di attivazione influenzino le prestazioni del modello, con effetti diversi a seconda della funzione di attivazione

utilizzata.

Per le configurazioni con singolo layer, l'aggiunta di neuroni (50, 100 o 200) ha comportato solo miglioramenti marginali nell'accuratezza. Le prestazioni non hanno mostrato miglioramenti significativi, e i tempi di addestramento sono rimasti piuttosto stabili. Le funzioni ReLU e tanh, pur essendo generalmente efficaci, non hanno tratto particolari vantaggi dall'aumento di neuroni o layer. Questo potrebbe essere dovuto al fatto che il dataset utilizzato è relativamente semplice e non richiede una rete complessa per identificare i pattern nei dati. Di conseguenza, la capacità del modello potrebbe essere già sufficiente a catturare le informazioni necessarie, portando a una saturazione in cui l'aggiunta di neuroni o layer non porta ad un effettivo miglioramento delle prestazioni, ma solo a un aumento del tempo di addestramento con l'incremento della profondità della rete.

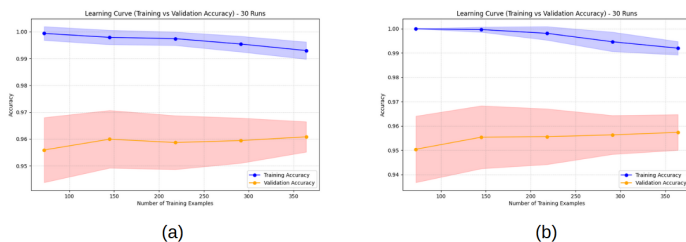


Figura 2: Curve di apprendimento per la funzione di attivazione identity al variare delle configurazioni dei layers: (a) layers (50), (b) layers (200,100,50)

Per la funzione identity, l'aggiunta di più neuroni o di un secondo layer ha portato a miglioramenti nell'accuratezza, con una media di 0.9825 nella configurazione 50 neuroni. Tuttavia, con l'introduzione di un terzo layer (200,100,50), l'accuratezza è diminuita a 0.9561, suggerendo un sovradimensionamento della rete, che potrebbe aver portato a overfitting. Ovvero un adattamento eccessivo della rete ai dati di addestramento, riducendo la sua capacità di generalizzazione sui dati di validazione. Interessante è il fatto che i tempi di addestramento sono rimasti stabili attorno a 0.3 secondi per tutte le configurazioni, indicando che la funzione identity non soffre di un incremento esponenziale nei tempi di calcolo con l'aumento della profondità della rete.

Per la funzione logistic, l'aggiunta di un secondo layer ha comportato dei miglioramenti nelle performance di training, ma ha anche causato un leggero calo nella validation accuracy (da 0.97 a 0.96). Questo suggerisce che, sebbene la rete si adatti meglio ai dati di addestramento, perde capacità di generalizzazione. L'introduzione di un terzo layer non ha portato miglioramenti, ma ha fatto aumentare notevolmente i tempi di addestramento: per un singolo layer il tempo medio è stato di circa 1 secondo, mentre con tre layer è salito a 3 secondi, indicando che la rete diventa più lenta e più difficile da ottimizzare con l'aumentare dei layer.

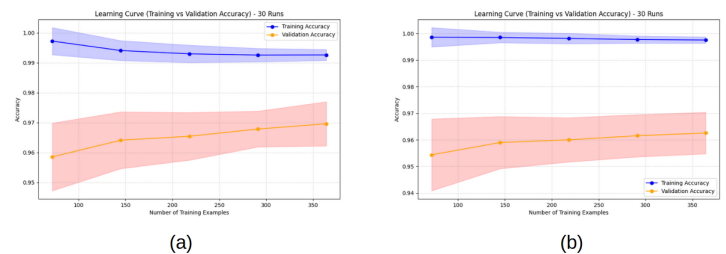


Figura 3: Curve di apprendimento per la funzione di attivazione logistic al variare delle configurazioni dei layers: (a) layers (50), (b) layers (200,100,50)

Learning rates iniziali, learning policies e solver
L'analisi delle prestazioni della rete neurale ha mostrato come diversi solver, learning rates iniziali e learning rate policies influenzino in modo significativo i risultati. Il solver **lbfgs**, un algoritmo quasi-newtoniano, ha garantito performance costanti con un'accuratezza media di circa 0.97 sulle 30 run. Questo solver converge rapidamente grazie alla sua efficienza nelle ottimizzazioni per dataset di dimensioni moderate, mostrando tempi di esecuzione rapidissimi (circa 0.1s) indipendentemente dal learning rate iniziale o dalla learning rate policy utilizzata.

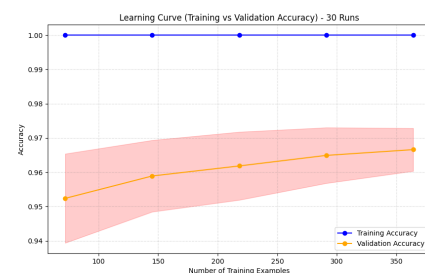


Figura 4: Curve di apprendimento per il solver lbfgs

Al contrario, il solver **adam**, che utilizza gradienti adattivi, ha mostrato un comportamento più sensibile al learning rate iniziale. Con valori più bassi, le prestazioni di validazione risultano leggermente superiori rispetto a lbfgs, accompagnate però da tempi di addestramento più elevati (oltre 1s) e da learning curves che evidenziano una convergenza più lenta in training. Tuttavia, variando la learning rate policy, adam non ha mostrato differenze significative, poiché il suo meccanismo di aggiornamento adattivo tende a compensare tali variazioni.

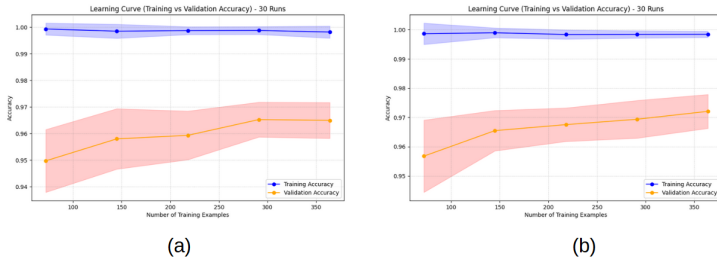


Figura 5: Curve di apprendimento per il solver **adam** al variare del learning rate iniziale: (a) learning rate iniziale = 0.1, (b) learning rate iniziale = 0.0001

Il solver **sgd** ha manifestato prestazioni meno stabili, particolarmente influenzate dal learning rate iniziale. Learning rates più alti hanno prodotto risultati migliori, mentre valori più bassi hanno causato un progressivo peggioramento delle performance. Tra le learning rate policies, adaptive e constant hanno mantenuto un comportamento simile, con un calo delle performance al diminuire del learning rate. La policy invscaling, invece, ha mostrato i peggiori risultati, con un'accuracy media di 0.5260 per un learning rate iniziale di 0.0001 e learning curves vicine al comportamento casuale (intorno a 0.5), sintomo di una convergenza insufficiente. Questo comportamento deriva dal fatto che sgd, a differenza di adam e lbfgs, richiede un tuning accurato per evitare oscillazioni o convergenza lenta, specialmente con policy che riducono troppo rapidamente il learning rate.

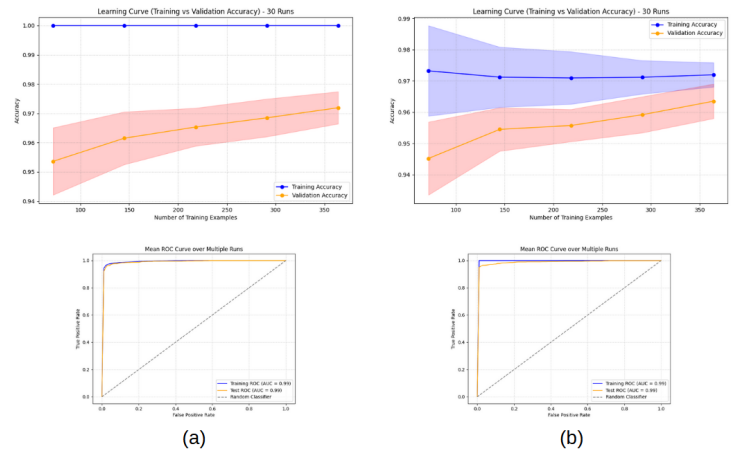


Figura 6: Curve di apprendimento e curve ROC per il solver **sgd** con learning rate policy adaptive al variare del learning rate iniziale: (a) learning rate iniziale = 0.1, (b) learning rate iniziale = 0.0001

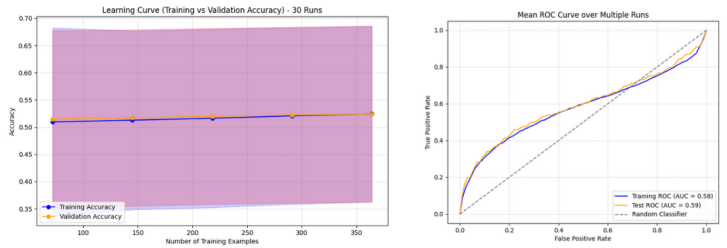


Figura 7: Curve di apprendimento e curve ROC per il solver **sgd** con learning rate policy invscaling

Batch size L'analisi dell'impatto del batch size sulle prestazioni della rete neurale ha evidenziato come dimensioni diverse influenzino significativamente sia la velocità di addestramento che la qualità delle prestazioni. Batch size piccoli, come 32 e 64, comportano tempi di addestramento più lunghi, ma garantiscono aggiornamenti più frequenti dei pesi, favorendo una convergenza più stabile e accurata, seppur con una maggiore variabilità nelle curve di apprendimento. Al crescere del batch size, il peggioramento più evidente si manifesta principalmente sul training set: batch più grandi, come 128 e 256, producono gradienti più precisi ma aggiornamenti meno frequenti, riducendo la capacità del modello di adattarsi ai dettagli dei dati di addestramento. Questo comporta una minore precisione sul training set, poiché il modello perde parte della sua capacità di cogliere le sottigliezze dei dati. Tuttavia, le prestazioni sul set di validazione tendono a essere più stabili grazie a una minore varia-

bilità nei gradienti. L'uso di batch size medi (64-128) si è rivelato un compromesso efficace, bilanciando velocità di addestramento e accuratezza, limitando la perdita di precisione sul training e mantenendo buone prestazioni sul set di validazione.

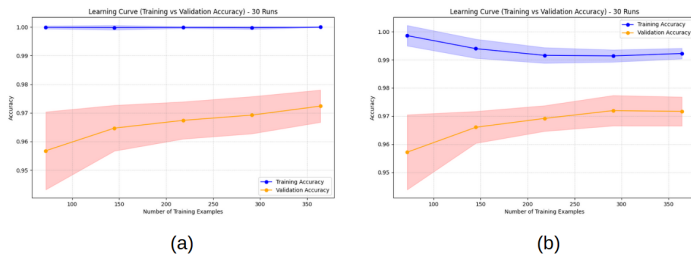


Figura 8: Curve di apprendimento e curve ROC per il solver *sgd* con learning rate policy *invscaling*

Parametri α e $n_iter_no_change$ L'analisi ha mostrato che la modifica del parametro di regolarizzazione α e del criterio di interruzione $n_iter_no_change$ non ha influenzato le prestazioni della rete neurale. L'assenza di variazioni al variare di α indica che il modello non soffre di overfitting significativo, rendendo la regolarizzazione poco rilevante. Inoltre, l'assenza di effetti legati a $n_iter_no_change$ suggerisce che la rete converge prima di raggiungere il limite massimo di iterazioni, indipendentemente dal valore impostato.

Early stopping

L'early stopping ha un effetto significativo sulle curve di apprendimento della rete neurale. Rispetto all'addestramento senza questa tecnica, si osserva un netto peggioramento della curva

di apprendimento del training, poiché l'aggiornamento dei pesi si interrompe prima del raggiungimento della convergenza. Questo porta a una diminuzione dell'accuracy sul training set, con un effetto più marcato per un numero ridotto di esempi di training. Anche la curva di apprendimento della validazione mostra un leggero peggioramento per set di training più piccoli, probabilmente a causa della minore capacità della rete di adattarsi ai dati. Tuttavia, un aspetto positivo dell'introduzione dell'early stopping è che le due curve di apprendimento (training e validation) risultano molto più vicine tra loro, segnalando una riduzione del rischio di overfitting.

Un ulteriore beneficio dell'early stopping è la drastica riduzione del tempo di training, che può diminuire fino a 10 volte, passando da una media di 0.6s a 0.06s. Questo avviene perché la rete interrompe l'addestramento non appena rileva che la performance sul set di validazione non migliora più, evitando iterazioni inutili.

Il validation split definisce la percentuale di dati di training da riservare alla validazione, influenzando quindi il punto in cui viene attivato l'early stopping. Aumentando questo valore, entrambe le curve di apprendimento si spostano verso l'alto, raggiungendo valori di accuracy più elevati. Questo potrebbe essere dovuto al fatto che con una porzione maggiore di dati riservata alla validazione, l'early stopping prende decisioni basate su un set più rappresentativo, favorendo un migliore bilanciamento tra training e validazione.

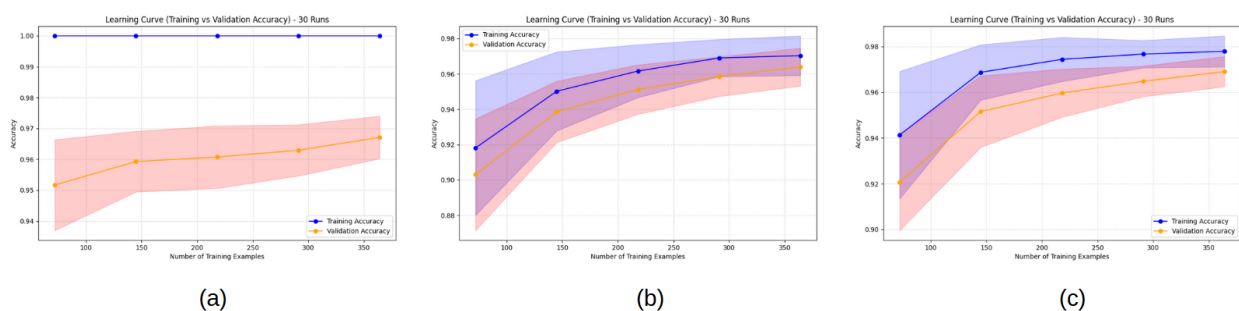


Figura 9: Curve di apprendimento: (a) Senza Early stopping, (b) Early stopping, validation split = 0.1, (c) Early stopping, validation split = 0.3